Primitive Ring Enumeration Module Specification

Revised S4R Design: Periodic Shortest-Path-Pair Primitive Rings

mdstats

2026-07-17 - implemented S4R with bounded periodic completeness proof

Contents

1 Purpose and status **3**

2 Design motives **4**

3 Algorithmic provenance and attribution **4**

4 Responsibility boundaries **4**
4.1 Owned by `primitive_ring.py` . . . 4
4.2 Not owned by this module . . . 5

5 Terminology **5**

6 Ring families **6**
6.1 Primitive no-shortcut family . . . 6
6.2 Edge-shortest subset . . . 6

7 Core correctness theory **7**
7.1 Graph setting . . . 7
7.2 Primitive and no-shortcut equivalence . . . 7
7.3 Maximal-half-arc sufficiency . . . 7
7.4 Translation-orbit representative lemma . . . 7
7.5 Even candidate representation . . . 8
7.6 Odd candidate representation . . . 8
7.7 Finite induced reduction . . . 8
7.7.1 Ring containment . . . 8
7.7.2 Shortcut-witness containment . . . 9
7.7.3 Transfer of the finite primitive theorem . . . 9
7.8 Bounded periodic completeness theorem . . . 9
7.9 External-shortcut reduction . . . 10

8 Input contract **10**
8.1 Required input . . . 10
8.2 Required topology invariants . . . 10
8.3 Additional assumptions . . . 10

9 Public enums, constants, and exceptions **11**

10 Public data structures **11**
10.1 `PrimitiveRingOptions` . . . 11
10.2 `LiftedVertexRef` . . . 12
10.3 `LiftedVertexPair` . . . 12
10.4 `PrimitiveRingStep` . . . 12

10.5 `PrimitiveRingEdgeToken` . . . 12
10.6 `PrimitiveRingKey` . . . 12
10.7 Internal candidate contract . . . 13
10.8 `PrimitiveShortcutWitness` . . . 13
10.9 `PrimitiveRing` . . . 13
10.10 Search diagnostics . . . 13
10.11 `PrimitiveRingSizeCount` . . . 14
10.12 `PrimitiveRingCatalog` . . . 14
10.13 `LiftedAtomRef` . . . 15

11 Public functions **15**
11.1 `enumerate_primitive_rings` . . . 15
11.2 `expand_primitive_ring_atomic_walk` . . . 16
11.3 Serialization . . . 16

12 Periodic graph model **16**

13 Bounded lifted shortest-path index **17**
13.1 Depth . . . 17
13.2 Sources . . . 17
13.3 Stored information . . . 17
13.4 Resource behavior . . . 17

14 Candidate generation **17**
14.1 Even cycles . . . 17
14.1.1 Two-member rings . . . 18
14.2 Odd cycles . . . 18
14.2.1 Triangles . . . 18
14.3 Early canonicalization . . . 18

15 Primitive no-shortcut classification **19**
15.1 Even cycles . . . 19
15.2 Odd cycles . . . 19
15.3 Acceptance query . . . 19

16 Optional external-shortcut witness **19**

17 Secondary removed-edge method **20**

18 Structural candidate validation **20**

19 Canonicalization and deterministic identity **20**

20 Multigraph and periodic edge cases **21**
20.1 Parallel edges . . . 21
20.2 Zero-shift self-edge . . . 21
20.3 Nonzero self-image edge . . . 21
20.4 Noncontractible loops . . . 21
20.5 Disconnected graphs . . . 21
20.6 High tied-path multiplicity . . . 21

21 Resource limits and transactional behavior **21**
21.1 Source-state limit . . . 21
21.2 Per-target path limit . . . 22
21.3 Path-pair combination limit . . . 22
21.4 Global candidate and ring limits . . . 22
21.5 Completeness language . . . 22

22 Complexity **22**

23 Atomic-path expansion **23**

24 Serialization and compatibility **23**

25 Validation plan **24**
25.1 Core analytical fixtures . . . 24
25.1.1 Triangle . . . 24
25.1.2 Square . . . 24
25.1.3 Square with a diagonal . . . 24
25.1.4 Parallel-edge pair . . . 24
25.1.5 Theta graph with unequal paths . . . 24
25.1.6 Octagon with short adjacent-edge detours . . . 24
25.1.7 Tree . . . 25
25.2 Periodic fixtures . . . 25
25.3 Decorated-path fixtures . . . 25
25.4 Certified-pair fixtures . . . 25
25.5 Resource fixtures . . . 25
25.6 Determinism fixtures . . . 25
25.7 Na-LTA gate . . . 26

26 Scientific warnings **26**

27 Revised implementation plan **26**
27.1 S4R.0 - API and terminology correction - complete . . . 26
27.2 S4R.1 - Lifted shortest-path index - complete . . . 26
27.3 S4R.2 - Parity-specific candidate generators - complete . . . 27
27.4 S4R.3 - Primitive classification - complete . . . 27
27.5 S4R.4 - Catalog construction and compatibility - complete . . . 27
27.6 S4R.5 - Validation and release gate - complete . . . 27

28 Implementation review checklist **27**

29 Normative summary **27**

30 References **28**

1 Purpose and status

This document specifies the **revised Stage 4 primitive-ring design** for

`mdstats/analysis/primitive_ring.py`

This specification is implemented by `mdstats 0.18.1a0` and retained in the current package. It defines the public API, correctness conditions, resource semantics, compatibility behavior, and validation requirements for the corrected primitive-ring foundation.

The `mdstats 0.18.0a0` removed-edge shortest-closure algorithm remains available as an explicitly selected **edge-shortest subset** method. It is useful but does not define the default primitive-ring family.

The revised default pipeline is

$$G_{\text{framework}} \xrightarrow{\text{bounded lifted shortest-path index}} \text{shortest-half-path candidates} \xrightarrow{\text{no-shortcut test}} \mathcal{R}_{\text{primitive}}.$$

The manual-review gate has passed. This document now records the implemented contract and the validation evidence required for future maintenance.

2 Design motives

The correction is required because the removed-edge method can miss primitive rings. If an edge belongs to a smaller primitive ring and a larger primitive ring, removing that shared edge exposes only the shorter replacement path. The larger ring is not generated even when it cannot be decomposed into two smaller rings.

A representative three-path arrangement is

$$|A| < |B| < |C|,$$

where paths A, B, and C share the same endpoints and are otherwise internally disjoint. The cycles $A \cup B$ and $A \cup C$ may both be primitive, while a removed-edge search rooted on A returns only $A \cup B$.

The revised design therefore separates two questions:

1. **Candidate generation:** Which bounded zero-winding cycles can be assembled from shortest half-paths?
2. **Primitive classification:** Does any pair of ring vertices admit a strict shortcut shorter than both cycle arcs between them?

The algorithm must also preserve the existing periodic and chemical constraints:

- exact lifted vertex images;
- zero periodic winding;
- decorated multigraph edge identity;
- asymmetric linker-path orientation;
- parallel projected edges;
- deterministic canonicalization;
- explicit search limits and completeness statements.

3 Algorithmic provenance and attribution

The revised generator is informed by several lines of published work:

- **Horton (1987)** introduced shortest-path candidate constructions for minimum cycle bases.
- **Vismara (1997)** showed how relevant cycles can be represented through a polynomial family of shortest-path prototypes and described parity-dependent shortest-path constructions.
- **Goetzke and Klein (1991)** analyzed classes of rings in finite and infinite polyhedral networks.
- **Yuan and Cormack (2002)** emphasized that shortest-path ring analysis does not contain all primitive rings and developed an efficient primitive-ring algorithm for topological networks, including periodic systems.

The `mdstats` algorithm is not a verbatim implementation of any one paper. It is a bounded periodic decorated-multigraph adaptation with explicit lifted images, exact physical edge instances, deterministic serialization, and resource semantics specific to this package.

The implementation must include concise author attribution in:

- the module docstring;
- comments adjacent to the shortest-path-pair candidate generator;
- comments adjacent to the no-shortcut classifier;
- the user-facing specification and architecture manual.

Full references appear at the end of this document.

4 Responsibility boundaries

4.1 Owned by `primitive_ring.py`

The module owns:

- bounded lazy exploration of the lifted periodic framework graph;

- a reusable all-source bounded shortest-path index;
- tied-shortest-path predecessor DAGs;
- parity-specific primitive-cycle candidate generation;
- propagation of certified shortest-pair provenance;
- the primitive no-shortcut test;
- optional external-shortcut witnesses;
- the legacy removed-edge shortest-closure search;
- lifted simple-cycle and physical-edge-instance validation;
- zero-winding validation;
- decorated-edge canonicalization;
- deterministic ring IDs and incidence indexes;
- search diagnostics, truncation records, and bounded completeness statements;
- schema-checked serialization and stable digests;
- orientation-aware expansion into lifted atomic paths.

4.2 Not owned by this module

The module must not:

- infer atomic connectivity from coordinates;
- construct or modify a `FrameworkTopology`;
- compute Cartesian centers, normals, areas, apertures, or puckering;
- identify cages, portals, channels, or adsorption sites;
- assert that primitive rings form a unique face tiling;
- equate primitive rings with physical pore windows;
- compute ring statistics across frames or topology classes;
- use NetworkX objects as canonical state.

The dependency direction remains

```
AtomicConnectivityState
          |
          v
FrameworkTopology
          |
          v
PrimitiveRingCatalog
          |
          +--> ring incidence / ring graph
          +--> ring geometry
          +--> cage and portal inference
          +--> ring-site assignment
          +--> ring statistics
```

5 Terminology

- **Quotient graph:** the finite decorated periodic framework representation in one reference cell.
- **Lifted graph:** the infinite periodic covering graph with explicit `(vertex_atom_index, lattice_image)` vertices.
- **Lifted vertex:** one quotient vertex plus an integer image vector.
- **Physical edge instance:** one translated copy of a quotient framework edge.
- **Shortest-path index:** bounded distances and tied-predecessor records from reference-image source vertices to lifted targets.
- **Primitive ring:** a local simple zero-winding cycle with no strict shortcut shorter than both cycle arcs between any two cycle vertices.
- **Edge-shortest ring:** a ring for which at least one removed physical edge is recovered by a tied shortest replacement path. This is a subset family, not the default primitive definition.

- **Maximal half-cycle pair:** a pair of cycle vertices separated by $\lfloor k/2 \rfloor$ edges along the shorter cycle arc.
- **Certified shortest pair:** a vertex pair already proved shortest by candidate construction and therefore omitted from later oracle queries.
- **Shortcut witness:** an external detour proving that a candidate is nonprimitive.
- **Framework ring size:** the number of projected framework vertices and edges in the cycle.
- **Winding vector:** net lattice translation accumulated around a cycle.
- **Complete edge identity:** the full decorated `FrameworkEdgeKey`, including endpoints, translation, ordered linker identities, linker image offsets, and rule identity.

6 Ring families

6.1 Primitive no-shortcut family

For a simple lifted cycle C and two vertices $x, y \in C$, let the two cycle arcs have lengths

$$\ell_1(x, y), \qquad \ell_2(x, y),$$

with

$$\ell_1 + \ell_2 = |C|.$$

The cycle is primitive when no graph path is strictly shorter than both arcs:

$$\boxed{d_{\widetilde{G}}(x, y) \geq \min\{\ell_1(x, y), \ell_2(x, y)\} \quad \text{for all } x, y \in C.}$$

Because the shorter cycle arc is itself a path in the graph,

$$d_{\widetilde{G}}(x, y) \leq \min\{\ell_1, \ell_2\}.$$

The practical acceptance condition is therefore equality:

$$d_{\widetilde{G}}(x, y) = \min\{\ell_1, \ell_2\}.$$

6.2 Edge-shortest subset

The secondary removed-edge method accepts cycles satisfying

$$\exists e \in C : \quad C \setminus e \text{ is a shortest replacement path in } \widetilde{G} - e.$$

The family relation is

$$\mathcal{R}_{\text{edge-shortest}} \subseteq \mathcal{R}_{\text{primitive}},$$

but equality is not guaranteed.

The secondary result must therefore report

`ring_family = "edge_shortest_subset"`

and must never claim complete primitive enumeration.

7 Core correctness theory

7.1 Graph setting

Let $\widetilde{G}$ be the infinite lifted decorated framework graph, let $\Lambda \cong \mathbb{Z}^d$ be its translation group, and let

$$G = \widetilde{G}/\Lambda$$

be the finite quotient. The graph is assumed locally finite, undirected, and unweighted. It may be a multigraph: physical parallel edges retain distinct identities.

A lifted vertex is

$$(u, \mathbf{n}), \qquad \mathbf{n} \in \mathbb{Z}^d.$$

A local ring is a lifted-simple finite cycle with zero total lattice winding.

7.2 Primitive and no-shortcut equivalence

The materials-network literature defines a primitive ring as a cycle that cannot be expressed as the symmetric-difference sum of two smaller cycles [3, 4]. For an unweighted graph, this is equivalent to the absence of a strict shortcut between ring vertices.

For $x, y \in C$, let $d_C(x, y)$ denote the shorter cycle-arc length. Then

$$C \text{ is primitive} \quad \Longleftrightarrow \quad d_{\widetilde{G}}(x, y) = d_C(x, y) \quad \text{for all } x, y \in C.$$

All sums and path comparisons use exact physical lifted edge instances.

7.3 Maximal-half-arc sufficiency

Let

$$r = \left\lfloor \frac{k}{2} \right\rfloor.$$

Every shorter arc of a k-cycle is a subpath of at least one length-r cycle arc. A subpath of a shortest path is shortest. Therefore, it is sufficient to verify the maximal half-cycle pairs.

For $k = 2r$, check the r antipodal pairs

$$(c_i, c_{i+r}), \qquad i = 0, \dots, r-1.$$

For $k = 2r + 1$, check the k pairs

$$(c_i, c_{i+r}), \qquad i = 0, \dots, k-1.$$

7.4 Translation-orbit representative lemma

Let C be any finite lifted cycle and let $(u, \mathbf{n})$ be one of its vertices. Translation by $-\mathbf{n}$ produces an equivalent cycle containing $(u, \mathbf{0})$. Translation is a graph automorphism and preserves:

- cycle size;
- lifted simplicity;
- graph distance;
- zero winding;

- shortcut and primitive status; and
- physical edge identity up to a common lattice translation.

Thus every translation orbit of finite lifted rings has a representative rooted at one of the quotient vertices in the base image.

7.5 Even candidate representation

If a primitive cycle has size $k = 2r$, choose antipodal lifted vertices u, v. The two u-v ring arcs have length r. Primitiveness forbids a shorter path, so both arcs are tied shortest paths:

$$d_{\widetilde{G}}(u, v) = r.$$

Every even primitive ring is therefore generated as the union of two internally lifted-vertex-disjoint shortest paths of length r between exact lifted endpoints.

7.6 Odd candidate representation

If a primitive cycle has size $k = 2r + 1$, choose a root u and the opposite lifted edge (v, w). The two ring paths

$$P_v : u \to v, \qquad P_w : u \to w$$

have length r and are shortest. Every odd primitive ring is therefore generated from two internally disjoint shortest root paths plus the exact closing edge instance (v, w).

No second-shortest-path oracle is required.

7.7 Finite induced reduction

For maximum requested ring size K, define

$$S = \{(u, \mathbf{0}) : u \in V(G)\}$$

and

$$H_K = \widetilde{G}\big[\{x : d_{\widetilde{G}}(x, S) \le K\}\big].$$

Because G is finite and $\widetilde{G}$ is locally finite, H_K is finite.

7.7.1 Ring containment

Translate a cycle C of size $k \le K$ so that it contains a vertex in S. Every vertex of C is reachable from that base vertex along the shorter cycle arc of length at most $\lfloor k/2 \rfloor$. Hence

$$C \subseteq H_K.$$

In fact, radius $\lfloor K/2 \rfloor$ contains the ring itself.

7.7.2 Shortcut-witness containment

Let P be a strict shortcut between ring vertices x, y. If a is the shorter cycle-arc length, then

$$|P| < a \leq \left\lfloor \frac{k}{2} \right\rfloor.$$

For any $z \in P$,

$$d_{\widetilde{G}}(z, S) \leq d_{\widetilde{G}}(x, S) + d_P(x, z) < k \leq K.$$

Therefore every strict-shortcut witness relevant to a cycle through size K is contained in H_K.

7.7.3 Transfer of the finite primitive theorem

If C is decomposable into two smaller finite cycles in $\widetilde{G}$, their union lies in a finite subgraph. Applying the finite-graph primitive-ring equivalence there produces a strict shortcut. The preceding bound places that shortcut in H_K. Conversely, every shortcut in H_K is also a shortcut in $\widetilde{G}$ and constructs two smaller cycles by symmetric difference with the two ring arcs.

Thus, for every translated cycle through size K,

$$C \text{ is primitive in } \widetilde{G} \quad \Longleftrightarrow \quad C \text{ is primitive in } H_K.$$

If a finite-graph statement is formulated only for simple graphs, replace every physical edge instance $e = (u, v)$ by its own subdivided path

$$u - m_e - v.$$

This removes parallel edges while doubling all path and cycle lengths uniformly. It preserves strict-shortcut inequalities, physical edge identity, and symmetric-difference decompositions. Hence the reduction also covers the decorated multigraph used by `mdstats`.

The finite graph H_K is a proof device. The implementation does not need to materialize it.

7.8 Bounded periodic completeness theorem

Let

$$R = \left\lfloor \frac{K}{2} \right\rfloor.$$

Suppose the algorithm:

1. indexes every quotient vertex root $(u, \mathbf{0})$ through depth R;
2. retains all tied shortest predecessor paths;
3. considers every eligible even and odd path combination;
4. preserves exact lifted vertices and physical edge instances; and
5. terminates without resource truncation.

Then it generates every lifted-simple, zero-winding primitive cycle of size at most K, up to lattice translation. Canonicalization under cyclic rotation, orientation reversal, and translation produces one catalog identity per ring orbit.

Accordingly, an untruncated default result may state:

> Complete for all lifted-simple, zero-winding primitive-ring translation orbits in the requested size interval under the specified unweighted decorated framework model.

The periodic translation-orbit proof and finite-radius reduction are original `mdstats` derivations. References [3, 4] provide the primitive-ring definitions and finite/infinite topological-network context.

7.9 External-shortcut reduction

If a violating path initially follows cycle edges, leaves the cycle, and later returns, remove its shared cycle prefix and suffix. If it touches the cycle at additional intermediate vertices, split it between successive cycle contacts. At least one resulting external segment remains shorter than the corresponding cycle arc.

Therefore every nonprimitive cycle has a witness whose endpoints lie on the cycle, whose internal vertices lie outside the cycle, and which does not traverse a cycle physical edge instance. This lemma justifies optional constrained witness recovery; the default Boolean classifier uses the global distance index.

8 Input contract

8.1 Required input

`FrameworkTopology`

The public enumerator accepts one immutable topology:

```
def enumerate_primitive_rings(
    topology: FrameworkTopology,
    *,
    options: PrimitiveRingOptions | None = None,
) -> PrimitiveRingCatalog:
    ...
```

Ring search is performed once per unique framework-topology class, not once per trajectory frame.

8.2 Required topology invariants

The input must satisfy the existing `FrameworkTopology` contract:

- framework vertex atom indices are nonempty, sorted, and unique;
- all edge endpoints resolve to retained framework vertices;
- projected edges are sorted by complete canonical `FrameworkEdgeKey`;
- exact duplicate edge keys are absent;
- endpoint translations are integer image vectors;
- nonperiodic axes have zero translations;
- degree and connected-component metadata are consistent;
- ordered linker paths are valid under whole-path reversal;
- spectator and excluded atoms do not occur as projected vertices or linker-path internals;
- the topology digest and schema version are valid.

8.3 Additional assumptions

The first revised implementation assumes:

- unweighted framework edges;
- finite quotient topology;
- locally finite lifted graph;
- ring sizes bounded by explicit integer options;
- exact labeled decorated-edge identity rather than graph isomorphism;
- one-member zero-shift loops excluded by default.

Disconnected topologies are valid. Ring search runs independently in each component through the same source-root loop.

9 Public enums, constants, and exceptions

```
from enum import Enum


class PrimitiveRingSearchMethod(str, Enum):
    SHORTEST_PATH_PAIRS = "shortest_path_pairs"
    REMOVED_EDGE_SHORTEST = "removed_edge_shortest"


class PrimitiveRingFamily(str, Enum):
    PRIMITIVE_NO_SHORTCUT = "primitive_no_shortcut"
    EDGE_SHORTEST_SUBSET = "edge_shortest_subset"
```

The module defines:

```
CANONICAL_PRIMITIVE_RING_SCHEMA = "mdstats.primitive-ring.v2"
LEGACY_PRIMITIVE_RING_SCHEMA = "mdstats.primitive-ring.v1"
PRIMITIVE_RING_ALGORITHM_VERSION = "shortest-path-pairs-v1"
PRIMITIVE_RING_DIGEST_ALGORITHM = "sha256"
```

Public exceptions are:

```
class PrimitiveRingError(ValueError): ...
class PrimitiveRingInputError(PrimitiveRingError): ...
class PrimitiveRingSearchError(PrimitiveRingError): ...
class PrimitiveRingComplexityError(PrimitiveRingSearchError): ...
class PrimitiveRingSerializationError(PrimitiveRingError): ...
```

`PrimitiveRingComplexityError` is raised only when `strict=True` and a transactional search limit is reached. Non-strict searches return an explicitly truncated catalog instead.

10 Public data structures

All identity-bearing result objects are frozen, slot-based dataclasses. Arrays, when used, must be defensively copied and read-only.

10.1 `PrimitiveRingOptions`

```
@dataclass(frozen=True, slots=True)
class PrimitiveRingOptions:
    method: PrimitiveRingSearchMethod = (
        PrimitiveRingSearchMethod.SHORTEST_PATH_PAIRS
    )

    min_ring_size: int = 2
    max_ring_size: int = 12

    max_lifted_states_per_source: int = 250_000
    max_shortest_paths_per_target: int = 100_000
    max_path_pair_combinations_per_anchor: int = 1_000_000
    max_total_candidates: int = 1_000_000
    max_total_rings: int = 250_000

    generate_shortcut_witnesses: bool = False
    allow_one_member_rings: bool = False
    strict: bool = False
```

```
    # Deprecated v1 constructor aliases retained during migration:
    max_lifted_states_per_edge: int | None = None
    max_shortest_paths_per_edge: int | None = None
    strict_resource_limits: bool | None = None
```

Validation requirements:

- `min_ring_size >= 2`;
- `allow_one_member_rings` is a reserved v1 compatibility field and must remain `False` in schema v2;
- `max_ring_size >= min_ring_size`;
- all resource limits are positive;
- the selected method is a declared enum member;
- the removed-edge method must not relabel its family as complete primitive.

The displayed numeric defaults are the implemented `0.18.1a0` defaults. They are resource policy rather than ring identity and may change only through an explicit compatibility review.

10.2 LiftedVertexRef

```
@dataclass(frozen=True, order=True, slots=True)
class LiftedVertexRef:
    atom_index: int
    image_shift: tuple[int, int, int]
```

Two records with the same atom index but different image shifts are distinct lifted vertices.

10.3 LiftedVertexPair

```
@dataclass(frozen=True, order=True, slots=True)
class LiftedVertexPair:
    first: LiftedVertexRef
    second: LiftedVertexRef
```

Construction canonicalizes endpoint order. This type is used for certified shortest-pair provenance and shortcut diagnostics.

10.4 PrimitiveRingStep

```
@dataclass(frozen=True, slots=True)
class PrimitiveRingStep:
    edge_index: int
    orientation: int  # +1 or -1
```

The orientation is relative to the canonical decorated framework-edge orientation. `edge_index` is dense within the supplied `FrameworkTopology`.

10.5 PrimitiveRingEdgeToken

```
@dataclass(frozen=True, order=True, slots=True)
class PrimitiveRingEdgeToken:
    edge_key: FrameworkEdgeKey
    orientation: int
```

Canonical ring identity uses complete structural edge keys, not transient integer edge IDs.

10.6 PrimitiveRingKey

```
@dataclass(frozen=True, order=True, slots=True)
class PrimitiveRingKey:
    edge_tokens: tuple[PrimitiveRingEdgeToken, ...]
```

The key is the minimum over all cyclic rotations of the forward traversal and all cyclic rotations of the completely reversed traversal.

10.7 Internal candidate contract

The following object may remain private, but its invariants are normative:

```
@dataclass(frozen=True, slots=True)
class _PrimitiveCycleCandidate:
    steps: tuple[PrimitiveRingStep, ...]
    vertex_walk: tuple[LiftedVertexRef, ...]
    certified_shortest_pairs: tuple[LiftedVertexPair, ...]
    generator_kind: str
    generator_anchor: object
```

`certified_shortest_pairs` prevents re-querying shortest-path facts already proved by construction.

10.8 `PrimitiveShortcutWitness`

```
@dataclass(frozen=True, slots=True)
class PrimitiveShortcutWitness:
    endpoint_pair: LiftedVertexPair
    first_cycle_arc_length: int
    second_cycle_arc_length: int
    shortcut_steps: tuple[PrimitiveRingStep, ...]
    shortcut_vertices: tuple[LiftedVertexRef, ...]
    shortcut_length: int
```

A witness is optional and exists only for a rejected nonprimitive candidate when `generate_shortcut_witnesses=True`.

10.9 `PrimitiveRing`

```
@dataclass(frozen=True, slots=True)
class PrimitiveRing:
    ring_id: int
    size: int
    steps: tuple[PrimitiveRingStep, ...]
    vertex_walk: tuple[LiftedVertexRef, ...]
    winding: tuple[int, int, int]
    key: PrimitiveRingKey

    # Populated by the removed-edge compatibility method only.
    generator_edge_indices: tuple[int, ...] = ()

    generator_kinds: tuple[str, ...] = ()
    generator_anchor_count: int = 0
    digest_algorithm: str = "sha256"
    digest: str = ""
```

The repeated closing vertex is not stored twice. Every accepted ring has `winding == (0, 0, 0)`. Read-only v2 aliases expose `edge_steps`, `vertex_ids`, `vertex_images`, and `canonical_key`; the retained field names preserve source compatibility with 0.18.0a0.

10.10 Search diagnostics

```
@dataclass(frozen=True, slots=True)
class PrimitiveRingSourceSearch:
    source_atom_index: int
    maximum_depth: int
```

```
    complete_through_depth: int
    visited_lifted_state_count: int
    target_state_count: int
    predecessor_record_count: int
    truncated: bool
    message: str | None

@dataclass(frozen=True, slots=True)
class PrimitiveRingSearchDiagnostics:
    index_depth: int
    source_searches: tuple[PrimitiveRingSourceSearch, ...]

    even_anchors_considered: int
    odd_anchors_considered: int
    shortest_paths_enumerated: int
    path_pair_combinations_considered: int

    structural_candidates: int
    canonical_candidates: int
    rejected_nonprimitive: int
    duplicate_candidates: int

    removed_edge_searches: tuple[PrimitiveRingEdgeSearch, ...]
    shortcut_witnesses: tuple[PrimitiveShortcutWitness, ...]
    shortcut_witness_count: int

    truncated: bool
    messages: tuple[str, ...]
```

`PrimitiveRingEdgeSearch` is retained for the secondary removed-edge method and for backward-compatible deserialization of v1 catalogs. Its status enum is:

```
class PrimitiveRingSearchStatus(str, Enum):
    COMPLETE_FOUND = "complete_found"
    COMPLETE_NONE = "complete_none"
    STATE_LIMIT_EXCEEDED = "state_limit_exceeded"
    PATH_LIMIT_EXCEEDED = "path_limit_exceeded"
    CANDIDATE_LIMIT_EXCEEDED = "candidate_limit_exceeded"
    NOT_SEARCHED_GLOBAL_LIMIT = "not_searched_global_limit"
    INVALID_EDGE = "invalid_edge"
    NOT_APPLICABLE = "not_applicable"
```

For the default global-index method, dense compatibility records use `NOT_APPLICABLE`; source-level diagnostics are authoritative.

10.11 PrimitiveRingSizeCount

```
@dataclass(frozen=True, slots=True)
class PrimitiveRingSizeCount:
    ring_size: int
    ring_count: int
```

10.12 PrimitiveRingCatalog

```
@dataclass(frozen=True, slots=True)
class PrimitiveRingCatalog:
    topology_digest: str
    topology_graph_digest: str
```

```
    options: PrimitiveRingOptions

    search_method: PrimitiveRingSearchMethod
    ring_family: PrimitiveRingFamily

    rings: tuple[PrimitiveRing, ...]

    # Dense edge-key compatibility records. For the default method their
    # status is NOT_APPLICABLE; removed-edge diagnostics live here directly.
    edge_searches: tuple[PrimitiveRingEdgeSearch, ...]
    ring_size_counts: tuple[PrimitiveRingSizeCount, ...]
    vertex_atom_indices: tuple[int, ...]
    vertex_to_ring_ids: tuple[tuple[int, ...], ...]
    edge_to_ring_ids: tuple[tuple[int, ...], ...]

    diagnostics: PrimitiveRingSearchDiagnostics
    search_completed_without_resource_truncation: bool
    complete_for_ring_sizes_up_to: int

    canonical_schema_version: str
    digest_algorithm: str
    digest: str
```

`size_counts` is a read-only alias for `ring_size_counts`. For `SHORTEST_PATH_PAIRS`, an untruncated result claims bounded completeness for primitive zero-winding rings. For `REMOVED_EDGE_SHORTEST`, the bound applies only to the edge-shortest subset.

10.13 LiftedAtomRef

```
@dataclass(frozen=True, order=True, slots=True)
class LiftedAtomRef:
    atom_index: int
    image_shift: tuple[int, int, int]
```

This is used by orientation-aware atomic-path expansion and does not carry Cartesian coordinates.

11 Public functions

11.1 enumerate_primitive_rings

```
def enumerate_primitive_rings(
    topology: FrameworkTopology,
    *,
    options: PrimitiveRingOptions | None = None,
) -> PrimitiveRingCatalog:
    ...
```

Behavior:

1. validate topology and options;
2. dispatch on `options.method`;
3. for the default method, build one bounded lifted shortest-path index;
4. generate even and odd candidates for the requested size interval;
5. apply structural, winding, and primitive checks;
6. canonicalize early and deduplicate;
7. assign deterministic IDs after sorting canonical keys;
8. construct incidence indexes and diagnostics;
9. serialize provenance and digest.

11.2 `expand_primitive_ring_atomic_walk`

```
def expand_primitive_ring_atomic_walk(
    topology: FrameworkTopology,
    ring: PrimitiveRing,
) -> tuple[LiftedAtomRef, ...]:
    ...
```

The helper preserves the Stage 2 whole-path orientation contract and reconstructs the relation between normalized projected-edge gauge and raw atomic-path gauge. It must never independently reverse endpoint order and linker order.

11.3 Serialization

```
def primitive_ring_catalog_digest(
    payload: Mapping[str, object],
) -> str:
    ...
```

The helper removes any existing `digest` field and hashes canonical JSON. `PrimitiveRingCatalog.to_dict()` and `from_dict()` preserve exact identity, resource semantics, method, family, and schema version.

The v2 reader supports v1 catalogs conservatively by interpreting them as

```
search_method = "removed_edge_shortest"
ring_family = "edge_shortest_subset"
```

without claiming primitive completeness.

12 Periodic graph model

For a canonical quotient edge

$$e = (u, v, \mathbf{m}_e),$$

the lifted graph contains

$$(u, \mathbf{n}) \longleftrightarrow (v, \mathbf{n} + \mathbf{m}_e)$$

for every $\mathbf{n} \in \mathbb{Z}^3$. The oriented half-edge translations are

$$u \to v : +\mathbf{m}_e, \qquad v \to u : -\mathbf{m}_e.$$

The adjacency retains every decorated physical edge orbit independently. Parallel edges cannot be collapsed to endpoint pairs.

For arbitrary lifted vertices,

$$d_{\widetilde{G}}((u, \mathbf{a}), (v, \mathbf{b})) = d_{\widetilde{G}}((u, \mathbf{0}), (v, \mathbf{b} - \mathbf{a})).$$

This relative-image identity is why one bounded source index per quotient vertex is sufficient for all translated distance queries.

13 Bounded lifted shortest-path index

13.1 Depth

For maximum requested ring size K,

$$\boxed{R_{\text{index}} = \left\lfloor \frac{K}{2} \right\rfloor.}$$

Depth R_{index} is required because candidate generation needs tied shortest paths of exact length $r = \lfloor k/2 \rfloor$. A radius of $r - 1$ would suffice only for shortcut detection, not for construction.

13.2 Sources

Use each quotient framework vertex once as a representative lifted root:

$$(u, \mathbf{0}).$$

Every translated local ring has a representative in which one selected root lies in the reference image. The translation-orbit lemma and bounded periodic completeness theorem above make this coverage explicit. Canonicalization removes translation duplicates.

13.3 Stored information

For each source, bounded BFS stores:

- distance to every reached lifted state;
- all equal-distance predecessor half-edges;
- deterministic predecessor order;
- counts needed to detect path multiplicity before backtracking;
- state and predecessor diagnostics.

The lookup key is effectively

`(source_atom_index, target_atom_index, relative_image)`

because quotient atom IDs alone do not determine periodic distance.

13.4 Resource behavior

If a source exceeds `max_lifted_states_per_source`, the index is incomplete. Default non-strict behavior returns an explicitly truncated catalog with no bounded completeness claim beyond what can be proved. Strict behavior raises `PrimitiveRingComplexityError`.

14 Candidate generation

14.1 Even cycles

For ring size

$$k = 2r,$$

consider every reached exact lifted target v satisfying

$$d_{\widetilde{G}}(u, v) = r.$$

Enumerate all tied shortest paths of length r from root u to target v. For every unordered pair (P_1, P_2):

- require internal lifted-vertex disjointness;
- require distinct physical edge-instance sequences;
- combine P_1 with the reverse of P_2;
- validate exact closure and zero winding.

The pair

$$\{u, v\}$$

is certified shortest by construction.

14.1.1 Two-member rings

For $k = 2$, the generic construction reduces to two distinct one-edge shortest paths between the same exact lifted endpoints. This is the dedicated parallel-edge case and must preserve decorated edge identity.

14.2 Odd cycles

For ring size

$$k = 2r + 1,$$

choose a root u and one exact lifted physical edge instance (v, w) whose two endpoints satisfy

$$d_{\widetilde{G}}(u, v) = r, \qquad d_{\widetilde{G}}(u, w) = r.$$

Enumerate tied shortest paths

$$P_v : u \to v, \qquad P_w : u \to w.$$

For every pair:

- require internal lifted-vertex disjointness;
- require that neither path uses the closing physical edge instance;
- combine P_v, the exact edge (v, w), and the reverse of P_w;
- validate exact closure and zero winding.

The pairs

$$\{u, v\}, \qquad \{u, w\}$$

are certified shortest by construction.

14.2.1 Triangles

For $k = 3$, all three maximal pairs are direct edges of length one. A simple zero-winding triangle requires no additional primitive-distance query.

14.3 Early canonicalization

The same cycle may be generated from several roots, antipodal pairs, opposite edges, translated copies, traversal directions, and tied path combinations. After structural validation, canonicalize immediately and retain one candidate record per `PrimitiveRingKey` before optional expensive diagnostics.

Generator provenance is merged rather than used as identity.

15 Primitive no-shortcut classification

Let the candidate be

$$C = (c_0, c_1, \dots, c_{k-1}).$$

15.1 Even cycles

For $k = 2r$, required maximal pairs are

$$\{c_i, c_{i+r}\}, \qquad i = 0, \dots, r-1.$$

One pair, the generator endpoints $\{u, v\}$, is already certified. The number of new distance checks is

$$\boxed{r - 1 = \frac{k}{2} - 1}.$$

15.2 Odd cycles

For $k = 2r + 1$, required maximal pairs are

$$\{c_i, c_{i+r}\}, \qquad i = 0, \dots, k-1.$$

The generator certifies $\{u, v\}$ and $\{u, w\}$. Therefore

$$\boxed{k - 2}$$

new checks remain for $k \geq 5$. For a triangle, no new check is needed.

15.3 Acceptance query

The cycle itself supplies a path of length r for each required pair. The index needs only determine whether a shorter path exists:

$$d_{\widetilde{G}}(x, y) < r.$$

- If true for any required pair, reject as nonprimitive.
- If false for every required pair, accept as primitive.

After index construction, classification cost is $O(k)$ constant-time lookups per candidate, with the certified-pair reductions above.

16 Optional external-shortcut witness

When a candidate fails and `generate_shortcut_witnesses=True`, run a constrained search that:

- removes the candidate's exact lifted physical edge instances;
- forbids cycle vertices as internal states;
- allows cycle vertices only as shortcut endpoints;
- finds a path shorter than both cycle arcs for one violating pair.

This search is diagnostic only. It is not part of default Boolean classification.

The witness should make a rejection auditable without forcing repeated constrained searches for every accepted ring.

17 Secondary removed-edge method

The existing v1 kernel remains available through

```
PrimitiveRingOptions(
    method=PrimitiveRingSearchMethod.REMOVED_EDGE_SHORTEST,
)
```

Its required semantics remain:

- select one representative physical edge instance;
- remove only that copy;
- retain all translated copies of the same quotient edge;
- search between exact lifted endpoints;
- retain tied shortest replacement paths;
- close candidates with the removed edge;
- canonicalize and deduplicate.

The output must report

```
search_method = "removed_edge_shortest"
ring_family = "edge_shortest_subset"
```

The method is useful for fast exploratory analysis and regression comparison, but its output is not a complete primitive catalog.

18 Structural candidate validation

Every candidate from either method must satisfy:

1. **Size:** the number of lifted vertices equals the requested ring size.
2. **Continuity:** every oriented edge step connects consecutive lifted vertices.
3. **Lifted simplicity:** no lifted vertex repeats except final closure.
4. **Physical-edge simplicity:** no physical edge instance repeats.
5. **Zero winding:** the final lifted vertex equals the starting lifted vertex.
6. **Decorated-path consistency:** traversal signs reverse complete linker paths.
7. **Nonperiodic axes:** all image components remain zero on nonperiodic axes.
8. **Multigraph distinction:** parallel decorated edges remain distinct.

Quotient atom IDs may repeat at different images; lifted vertex identity is the criterion.

19 Canonicalization and deterministic identity

For oriented token sequence

$$S = [(e_0, \eta_0), \dots, (e_{k-1}, \eta_{k-1})],$$

construct:

- every cyclic rotation of S;
- the complete reverse
$$S^{-1} = [(e_{k-1}, -\eta_{k-1}), \dots, (e_0, -\eta_0)];$$
- every cyclic rotation of S^{-1}.

Choose the lexicographically smallest complete decorated token sequence.

Identity is invariant under:

- starting vertex;
- traversal direction;

- global lattice translation;
- generator root or anchor;
- discovery order.

Identity preserves:

- parallel edge distinctions;
- periodic translations;
- ordered linker paths;
- linker image offsets;
- rule identity;
- whole-path reversal coupling.

Unique keys are sorted before dense ring IDs are assigned.

20 Multigraph and periodic edge cases

20.1 Parallel edges

Two distinct parallel decorated edges may form a valid 2-ring. Endpoint equality alone must not collapse them.

20.2 Zero-shift self-edge

A zero-shift self-edge is a one-member cycle. Schema v2 does not enumerate it. `min_ring_size=1` and `allow_one_member_rings=True` are rejected explicitly; the field is retained only to parse legacy option payloads without silent behavior.

20.3 Nonzero self-image edge

An edge

$$(u, \mathbf{0}) \longleftrightarrow (u, \mathbf{m}), \qquad \mathbf{m} \neq \mathbf{0},$$

has distinct lifted endpoints and is processed normally.

20.4 Noncontractible loops

A quotient cycle with nonzero winding is not a local ring and is rejected even if its quotient vertex sequence closes.

20.5 Disconnected graphs

Each component is searched independently through the same source loop. No global connectedness requirement is imposed.

20.6 High tied-path multiplicity

Path enumeration and path-pair combination can be exponential in pathological graphs. Resource limits are mandatory and transactional.

21 Resource limits and transactional behavior

21.1 Source-state limit

If a source BFS exceeds `max_lifted_states_per_source`, that source index is incomplete.

21.2 Per-target path limit

Before explicit backtracking, count tied shortest paths through the predecessor DAG with saturation at

$$N_{\max} + 1.$$

If the count exceeds `max_shortest_paths_per_target`, do not enumerate a partial prefix and claim completeness.

21.3 Path-pair combination limit

For each even endpoint pair or odd root-edge anchor, count or transactionally track path-pair combinations. If the anchor exceeds `max_path_pair_combinations_per_anchor`, discard candidates from that incomplete anchor and record truncation.

21.4 Global candidate and ring limits

Candidate and final-ring limits prevent unbounded memory use. Crossing either limit marks the search incomplete or raises in strict mode.

21.5 Completeness language

For the default method, an untruncated result may state:

> Complete for all lifted-simple, zero-winding primitive-ring translation orbits in the requested size interval under the specified unweighted decorated framework model.

This claim is supported by the translation-orbit representative lemma, the even and odd shortest-path representations, and the finite induced reduction H_K.

For the removed-edge method, an untruncated result may state only:

> Complete for local zero-winding edge-shortest closure rings in the requested size interval.

No bounded search may claim that larger rings do not exist. Any truncated source, target path set, path-pair anchor, candidate set, or accepted-ring set invalidates the corresponding primitive completeness claim.

22 Complexity

Let:

- N be the number of quotient framework vertices;
- Δ be maximum lifted degree;
- K be maximum requested ring size;
- $R = \lfloor K/2 \rfloor$;
- B_R be the number of lifted states reached within radius R from one source;
- Q be the number of shortest-path-pair combinations actually considered.

The bounded all-source index costs approximately

$$O(NB_R)$$

in time and storage, plus predecessor multiplicity.

Candidate generation and reconstruction are output-sensitive:

$$O(QK).$$

After the index exists, primitive classification is

$$O(k)$$

per candidate, with lookup counts reduced to

$$\frac{k}{2} - 1$$

for even rings and

$$k - 2$$

for odd rings of size at least five.

For fixed K and bounded Δ, the structural neighborhood term scales approximately linearly with N. Explicit enumeration can still be exponential in pathological graphs because the number of tied shortest-path pairs or primitive rings may itself be exponential. The implementation must be output-sensitive and resource-bounded rather than claiming a universal polynomial explicit listing bound.

23 Atomic-path expansion

Ring topology remains coordinate-free. Atomic expansion reconstructs the ordered lifted path through framework vertices and linkers.

Projected endpoint shifts and raw atomic-path shifts use different deterministic gauges. The helper must reconstruct the per-framework-vertex gauge relation before combining them. Directly adding normalized projected shifts to raw linker shifts is invalid.

The expanded walk must:

- preserve complete linker order;
- reverse endpoint, linker, and translation order together;
- contain no spectator atoms;
- close at the exact starting lifted atom;
- remain independent of trajectory coordinates.

24 Serialization and compatibility

The v2 payload must include:

- topology digest;
- method and ring family;
- all options and resource limits;
- exact ring keys and traversal data;
- diagnostics and completeness fields;
- schema and algorithm versions;
- stable digest.

A v1 payload may be accepted by migration logic and relabeled as the edge-shortest subset. Migration must never silently upgrade a v1 catalog to complete primitive status.

Changing any of the following requires a schema or algorithm-version increment:

- primitive definition;
- candidate-generation parity rules;
- complete edge token identity;
- canonicalization;
- periodic gauge conventions;
- completeness semantics.

25 Validation plan

25.1 Core analytical fixtures

25.1.1 Triangle

Expected:

- one primitive 3-ring;
- no additional primitive lookups;
- zero winding;
- identical result under both methods.

25.1.2 Square

Expected:

- one primitive 4-ring;
- even generator uses two length-2 shortest paths;
- one certified antipodal pair and one remaining lookup.

25.1.3 Square with a diagonal

Expected:

- two primitive 3-rings;
- outer 4-cycle rejected by a strict shortcut.

25.1.4 Parallel-edge pair

Expected:

- one primitive 2-ring;
- edge identity remains decorated and multigraph-aware.

25.1.5 Theta graph with unequal paths

Construct three internally disjoint paths with lengths

$$|A| < |B| < |C|.$$

Expected:

- $A \cup B$ primitive;
- $A \cup C$ primitive when no additional shortcut exists;
- $B \cup C$ nonprimitive;
- removed-edge method may omit $A \cup C$;
- default method must recover both primitive cycles.

25.1.6 Octagon with short adjacent-edge detours

Construct an eight-cycle and add a distinct two-edge detour parallel to every octagon edge. The detours create eight triangles. They do not violate the primitive criterion for the octagon because each detour has length two and is not shorter than the adjacent one-edge cycle arc.

Expected:

- default shortest-path-pair method: eight 3-rings and one primitive 8-ring;
- removed-edge method: eight 3-rings and no 8-ring.

This fixture directly proves that iterating removed-edge shortest closures can miss a primitive cycle when every edge has a shorter replacement path.

25.1.7 Tree

Expected: no rings.

25.2 Periodic fixtures

The periodic test suite must include explicit bounded-completeness fixtures:

- translate every enumerated ring so that one vertex lies in the base image and verify that the same canonical identity is recovered;
- materialize a small finite induced ball H_K and compare its primitive cycles through K with the lazy lifted result;
- verify that all strict-shortcut witnesses for those cycles remain inside the radius-K ball;
- include a quotient graph whose ring uses multiple translated instances of one edge orbit; and
- verify primitive-cell and supercell translation-orbit equivalence.
- boundary-crossing zero-winding local ring;
- noncontractible nonzero-winding loop;
- equal quotient endpoints with different target images;
- translated duplicates of the same local cycle;
- nonzero self-image edge;
- parallel decorated periodic edges.

25.3 Decorated-path fixtures

Verify that

$$A - \mathrm{O} - \mathrm{S} - B \equiv B - \mathrm{S} - \mathrm{O} - A$$

under complete reversal, while

$$A - \mathrm{O} - \mathrm{S} - B \neq A - \mathrm{S} - \mathrm{O} - B.$$

25.4 Certified-pair fixtures

Instrument the classifier and verify:

- even k: exactly $k/2 - 1$ new lookups;
- odd $k \geq 5$: exactly $k - 2$ new lookups;
- triangle: zero new lookups;
- accepted/rejected results are unchanged if certified-pair optimization is disabled in a reference implementation.

25.5 Resource fixtures

Force every resource limit independently and verify:

- no partial anchor is represented as complete;
- strict mode raises;
- non-strict mode records truncation;
- digest includes truncation state.

25.6 Determinism fixtures

Randomize:

- quotient edge order;
- predecessor insertion order;
- source iteration order;

- tied-path enumeration order.

Require identical ring keys, IDs, incidence indexes, serialization, and digest.

25.7 Na-LTA gate

Run both methods on the uniform 300 K Na-LTA framework topology.

The implemented size-eight acceptance run gives:

```
primitive/no-shortcut: 36 x 4R + 40 x 6R + 6 x 8R
edge-shortest subset:   36 x 4R + 16 x 6R
```

Both searches complete without resource truncation. The six 8-cycles are the expected topological candidates missing from the removed-edge subset. The 40 primitive 6-cycles are not automatically identical to the 16 conventional 6-ring site families; downstream geometry and cage/portal classification must resolve that distinction. Extending the bound through size twelve also finds 32 primitive 12-cycles.

The acceptance report compares both families, primitive lookup counts, resource limits, atomic-path expansion, serialization, and deterministic repeatability.

26 Scientific warnings

1. Primitive rings are not automatically physical pore windows.
2. A physically important window may be primitive, nonprimitive, or represented by several topological cycle candidates depending on framework geometry.
3. Ring size is measured in projected framework vertices, not all atoms in the expanded linker walk.
4. The default algorithm is complete only within the requested size interval and only when no resource limit is exceeded.
5. The legacy removed-edge method is a subset method.
6. Distance queries must include exact lifted image displacement.
7. Equal quotient atom IDs do not imply equal lifted vertices.
8. Parallel edges and ordered linker paths are identity-bearing.
9. Geometry, cage boundaries, and site labels must not be inferred inside this module.
10. Published relevant-cycle, minimum-cycle-basis, and primitive-ring definitions are related but not interchangeable; the implementation must use the exact no-shortcut contract stated here.

27 Revised implementation plan

27.1 S4R.0 - API and terminology correction - complete

- add search-method and ring-family enums;
- advance serialization to v2;
- relabel v1 catalogs as edge-shortest subsets;
- update docstrings, references, and completeness language;
- preserve old method behind explicit options.

27.2 S4R.1 - Lifted shortest-path index - complete

- deterministic bounded all-source BFS;
- exact relative-image targets;
- tied-predecessor DAGs;
- saturated path counting;
- source-level diagnostics and limits;
- direct unit tests independent of cycle generation.

27.3 S4R.2 - Parity-specific candidate generators - complete

- even candidates from pairs of internally disjoint tied shortest paths;
- odd candidates from two shortest root paths plus one closing edge;
- dedicated 2-ring and triangle paths;
- certified shortest-pair provenance;
- early canonical deduplication;
- path-pair transaction limits.

27.4 S4R.3 - Primitive classification - complete

- maximal-half-cycle pair generation;
- certified-pair subtraction;
- global-index Boolean no-shortcut test;
- optional external-shortcut witness;
- synthetic shortcut, theta, and octagon-with-detours regressions.

27.5 S4R.4 - Catalog construction and compatibility - complete

- deterministic IDs and incidence indexes;
- method/family-aware serialization;
- v1 migration;
- atomic-path expansion regression;
- full provenance and digests.

27.6 S4R.5 - Validation and release gate - complete

- complete analytical and periodic suite;
- full-package regression;
- installed-wheel smoke test;
- Na-LTA comparison between methods;
- revised Markdown/PDF alignment review;
- audit documenting every stage and any unresolved ring-definition discrepancy.

S4R.5 has passed. The next dependent layer is the common periodic-graph refactor and `ring_complex.py`.

28 Implementation review checklist

The completed S4R release confirms that:

1. the no-shortcut definition is the default scientific primitive-ring family;
2. shortest-path-pair generation is the default method;
3. the removed-edge method is retained only as an explicitly labeled subset;
4. the even and odd constructions preserve exact periodic lifted identity;
5. no second-shortest-path oracle is used;
6. the shared index depth is $\lfloor K/2 \rfloor$;
7. certified-pair reductions avoid redundant primitive lookups;
8. 2-rings and triangles are handled explicitly by the parity generators;
9. resource limits are transactional and visible;
10. v1 migration relabels results only as edge-shortest subsets;
11. bounded periodic completeness follows from the translation-orbit and finite-radius proofs;
12. the next dependent layer is `ring_complex.py`, not a standalone ring-adjacency graph; and
13. published algorithms are distinguished from the package-specific periodic and decorated-edge adaptations.

29 Normative summary

The revised default contract is:

$$\boxed{\texttt{SHORTEST\_PATH\_PAIRS} \implies \texttt{PRIMITIVE\_NO\_SHORTCUT}}$$

with:

- one bounded lifted shortest-path index;
- even candidates from two shortest antipodal paths;
- odd candidates from two shortest root paths plus one closing edge;
- certified-pair provenance;
- maximal-half-cycle no-shortcut tests;
- optional external witness generation;
- exact periodic and decorated multigraph identity;
- explicit bounded periodic completeness from the translation-orbit and finite-radius proofs.

The secondary contract is:

$$\boxed{\texttt{REMOVED\_EDGE\_SHORTEST} \implies \texttt{EDGE\_SHORTEST\_SUBSET}}$$

and must never be described as complete primitive enumeration.

30 References

1. Horton, J. D. (1987). *A Polynomial-Time Algorithm to Find the Shortest Cycle Basis of a Graph.* SIAM Journal on Computing, 16(2), 358-366. DOI: 10.1137/0216026.
2. Vismara, P. (1997). *Union of All the Minimum Cycle Bases of a Graph.* The Electronic Journal of Combinatorics, 4(1), Research Paper R9. DOI: 10.37236/1294.
3. Goetzke, K., and Klein, H. J. (1991). *Properties and Efficient Algorithmic Determination of Different Classes of Rings in Finite and Infinite Polyhedral Networks.* Journal of Non-Crystalline Solids, 127, 215-220.
4. Yuan, X., and Cormack, A. N. (2002). *Efficient Algorithm for Primitive Ring Statistics in Topological Networks.* Computational Materials Science, 24(3), 343-360. DOI: 10.1016/S0927-0256(01)00256-7.
5. Guttman, L. (1990). *Ring Structure of the Crystalline and Amorphous Forms of Silicon Dioxide.* Journal of Non-Crystalline Solids, 116, 145-147.
6. Chung, S. J., Hahn, Th., and Klee, W. E. (1984). *Nomenclature and Generation of Three-Periodic Nets: The Vector Method.* Acta Crystallographica Section A, 40, 42-50. DOI: 10.1107/S0108767384000088.
7. Klee, W. E. (2004). *Crystallographic Nets and Their Quotient Graphs.* Crystal Research and Technology, 39(11), 959-968. DOI: 10.1002/crat.200410281.

References [1, 2] motivate shortest-path cycle prototypes. References [3-5] provide primitive-ring definitions and ring-statistics context. References [6, 7] provide the periodic quotient-graph framework. The bounded translation-orbit and finite-radius completeness proofs are original `mdstats` derivations.